

FINAL PROJECT REPORT

BANK ACCOUNT MANAGEMENT SYSTEM

Java Application Development
Week 6 Final Project

6 Week Java Development Internship
Technology: Java
Development Environment: Eclipse IDE
Repository: GitHub

V.Bhavana

1. Project Title

Bank Account Management System

Technology: Java | Type: Console-Based Application | IDE: Eclipse

2. Introduction

The Bank Account Management System is a console-based Java application developed to manage bank account information and basic banking operations. The application allows users to create Savings and Current accounts and perform operations such as viewing, searching, updating, deleting, depositing, and withdrawing money. The project was developed to apply the Java programming concepts learned during the 6-week Java Development Internship.

3. Objective

- To develop a functional Java-based management system.
- To understand and implement Object-Oriented Programming concepts.
- To manage bank account records using Java Collections.
- To handle invalid inputs and runtime errors using exception handling.
- To store and retrieve account information using file handling.
- To demonstrate basic multithreading.
- To gain practical experience in Git and GitHub.
- To develop a menu-driven console application.

4. Features

- Add a new bank account
- View all accounts
- Search an account by account number
- Update account holder name
- Delete an account
- Deposit money
- Withdraw money
- Savings Account support
- Current Account support with overdraft limit
- File handling for saving account information
- Reading saved account data
- Exception handling for invalid input
- Background transaction verification using multithreading
- Menu-driven console interface

5. Concepts Used

OOP Concepts:

- Classes and Objects – Used to represent accounts, transactions, and bank operations.
- Constructors – Used to initialize account objects.
- this keyword – Used in constructors and object initialization.
- Encapsulation – Account details are maintained using private fields and getter/setter methods.
- Inheritance – SavingsAccount and CurrentAccount inherit from Account.
- Polymorphism – Different account types are handled using the Account reference.
- Abstraction – Account is an abstract class containing the common account structure.
- static – Used for common methods and shared program resources.

Other Java Concepts:

- Exception Handling – try-catch is used to handle invalid input and file errors.
- Collections – HashMap is used to store accounts.
- ArrayList – Used to return a collection of all accounts.
- File Handling – FileWriter, PrintWriter, FileReader, and BufferedReader are used.
- Multithreading – TransactionTask extends Thread for background transaction verification.
- Packages – Application classes are organized under the bankmanagement package.

6. Project Structure

BankAccountManagementSystem

```

|
├── Account.java
├── SavingsAccount.java
├── CurrentAccount.java
├── Transaction.java
├── Bank.java
├── FileManager.java
├── TransactionTask.java
├── Main.java
├── accounts.txt
└── README.md

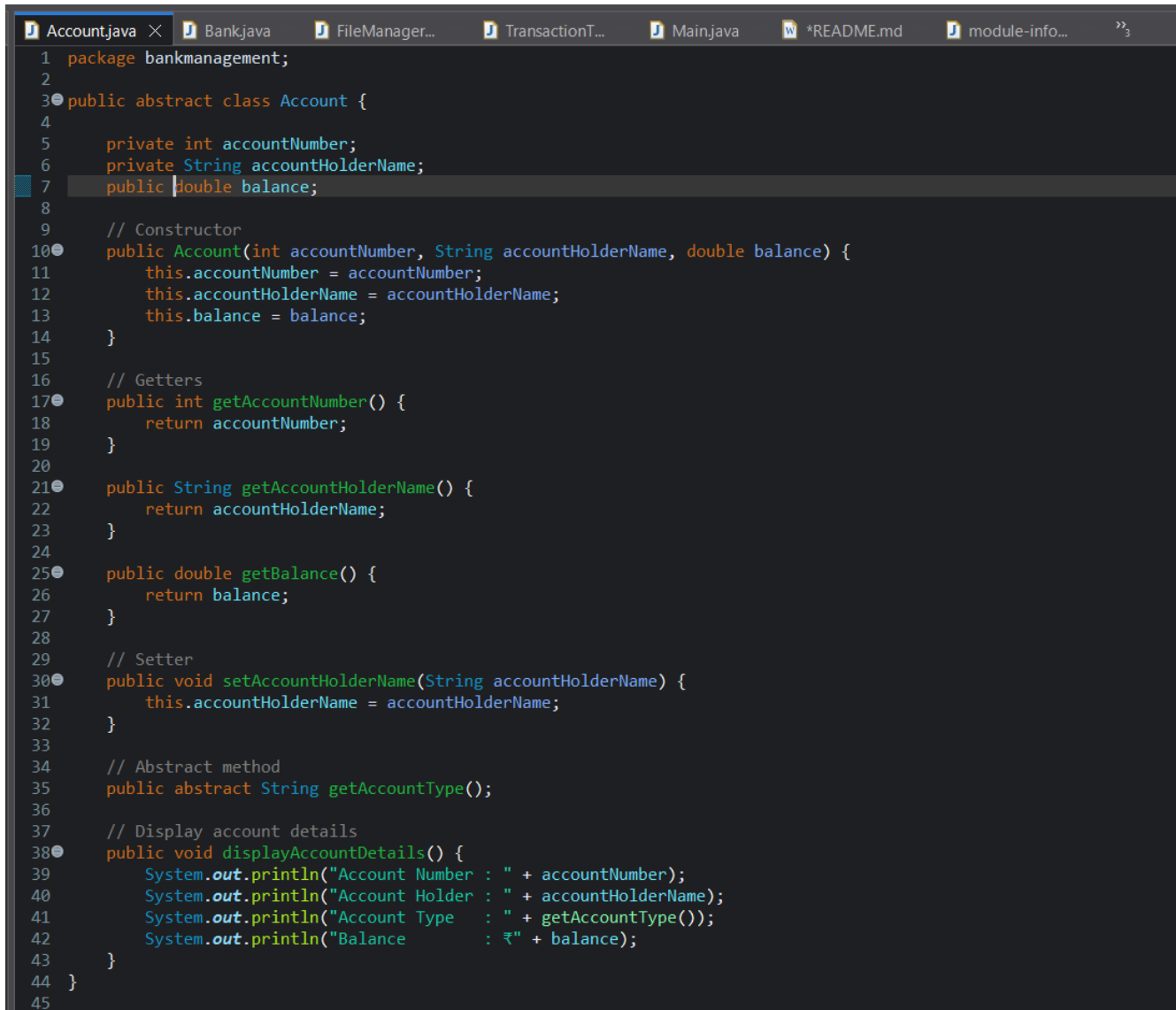
```

Class	Purpose
Account.java	Abstract base class for bank accounts
SavingsAccount.java	Handles Savings Account details
CurrentAccount.java	Handles Current Account and overdraft
Transaction.java	Handles deposit and withdrawal
Bank.java	Performs account management operations
FileManager.java	Saves and reads account data
TransactionTask.java	Demonstrates multithreading
Main.java	Provides the menu-driven interface

7. Code Screenshots

Insert screenshots from Eclipse showing the important sections of the source code. Recommended screenshots are:

1. Account.java – abstract class, fields, constructor, getters/setter, and abstract method.
2. SavingsAccount.java and CurrentAccount.java – inheritance and method overriding.
3. Bank.java – HashMap and Add/View/Search/Update/Delete operations.
4. FileManager.java – file writing and reading with exception handling.
5. TransactionTask.java – Thread implementation.



```
1 package bankmanagement;
2
3 public abstract class Account {
4
5     private int accountNumber;
6     private String accountHolderName;
7     public double balance;
8
9     // Constructor
10    public Account(int accountNumber, String accountHolderName, double balance) {
11        this.accountNumber = accountNumber;
12        this.accountHolderName = accountHolderName;
13        this.balance = balance;
14    }
15
16    // Getters
17    public int getAccountNumber() {
18        return accountNumber;
19    }
20
21    public String getAccountHolderName() {
22        return accountHolderName;
23    }
24
25    public double getBalance() {
26        return balance;
27    }
28
29    // Setter
30    public void setAccountHolderName(String accountHolderName) {
31        this.accountHolderName = accountHolderName;
32    }
33
34    // Abstract method
35    public abstract String getAccountType();
36
37    // Display account details
38    public void displayAccountDetails() {
39        System.out.println("Account Number : " + accountNumber);
40        System.out.println("Account Holder : " + accountHolderName);
41        System.out.println("Account Type : " + getAccountType());
42        System.out.println("Balance : ₹" + balance);
43    }
44 }
45
```

```

1 package bankmanagement;
2
3 import java.util.ArrayList;
4 import java.util.HashMap;
5
6 public class Bank {
7
8     // Collection to store accounts
9     private HashMap<Integer, Account> accounts;
10
11     // Constructor
12     public Bank() {
13         accounts = new HashMap<>();
14     }
15
16     // Add Account
17     public void addAccount(Account account) {
18
19         if (accounts.containsKey(account.getAccountNumber())) {
20             System.out.println("Account number already exists.");
21             return;
22         }
23
24         accounts.put(account.getAccountNumber(), account);
25         System.out.println("Account added successfully.");
26     }
27
28     // View All Accounts
29     public void viewAllAccounts() {
30
31         if (accounts.isEmpty()) {
32             System.out.println("No accounts available.");
33             return;
34         }
35
36         System.out.println("\n===== ALL ACCOUNTS =====");
37
38         for (Account account : accounts.values()) {
39             account.displayAccountDetails();
40             System.out.println("-----");
41         }
42     }
43
44     // Search Account
45     public Account searchAccount(int accountNumber) {

```

```
Account.java Bank.java × FileManager... TransactionT... Main.java *README.md module-info
38●   for (Account account : accounts.values()) {
39       account.displayAccountDetails();
40       System.out.println("-----");
41   }
42   }
43
44   // Search Account
45●   public Account searchAccount(int accountNumber) {
46
47       return accounts.get(accountNumber);
48   }
49
50   // Update Account Holder Name
51●   public void updateAccount(int accountNumber, String newName) {
52
53       Account account = accounts.get(accountNumber);
54
55●   if (account == null) {
56       System.out.println("Account not found.");
57       return;
58   }
59
60       account.setAccountHolderName(newName);
61
62       System.out.println("Account updated successfully.");
63   }
64
65   // Delete Account
66●   public void deleteAccount(int accountNumber) {
67
68       Account removedAccount = accounts.remove(accountNumber);
69
70●   if (removedAccount == null) {
71       System.out.println("Account not found.");
72●   } else {
73       System.out.println("Account deleted successfully.");
74   }
75   }
76
77   // Get all accounts
78●   public ArrayList<Account> getAllAccounts() {
79
80       return new ArrayList<>(accounts.values());
81   }
82   }
```

```
Account.java  CurrentAccou... ×  Bank.java  TransactionT...  Main.java  *README.m
1 package bankmanagement;
2
3 public class CurrentAccount extends Account {
4
5     private double overdraftLimit;
6
7     // Constructor
8     public CurrentAccount(int accountNumber, String accountHolderName,
9                           double balance, double overdraftLimit) {
10
11         super(accountNumber, accountHolderName, balance);
12         this.overdraftLimit = overdraftLimit;
13     }
14
15     // Getter
16     public double getOverdraftLimit() {
17         return overdraftLimit;
18     }
19
20     // Polymorphism
21     @Override
22     public String getAccountType() {
23         return "Current Account";
24     }
25
26     // Check withdrawal limit
27     public boolean canWithdraw(double amount) {
28         return amount <= balance + overdraftLimit;
29     }
30 }
```

```

1 package bankmanagement;
2
3 import java.io.BufferedReader;
4 import java.io.FileReader;
5 import java.io.FileWriter;
6 import java.io.IOException;
7 import java.io.PrintWriter;
8
9 public class FileManager {
10
11     private static final String FILE_NAME = "accounts.txt";
12
13     // Save accounts to file
14     public static void saveAccounts(Bank bank) {
15
16         try (PrintWriter writer = new PrintWriter(new FileWriter(FILE_NAME))) {
17
18             for (Account account : bank.getAllAccounts()) {
19
20                 String type = account.getAccountType();
21
22                 double extraValue = 0;
23
24                 if (account instanceof SavingsAccount) {
25                     SavingsAccount savings = (SavingsAccount) account;
26                     extraValue = savings.getInterestRate();
27
28                 } else if (account instanceof CurrentAccount) {
29                     CurrentAccount current = (CurrentAccount) account;
30                     extraValue = current.getOverdraftLimit();
31                 }
32
33                 writer.println(
34                     type + "|" +
35                     account.getAccountNumber() + "|" +
36                     account.getAccountHolderName() + "|" +
37                     account.getBalance() + "|" +
38                     extraValue
39                 );
40             }
41
42             System.out.println("Accounts saved to file successfully.");
43
44         } catch (IOException e) {
45             System.out.println("Error while saving accounts: " + e.getMessage());
46         }
47     }
48 }

```

```

44     } catch (IOException e) {
45         System.out.println("Error while saving accounts: " + e.getMessage());
46     }
47 }
48
49 // Read accounts from file
50 public static void readAccounts() {
51     try (BufferedReader reader = new BufferedReader(new FileReader(FILE_NAME))) {
52         String line;
53
54         System.out.println("\n===== SAVED ACCOUNT DATA =====");
55
56         while ((line = reader.readLine()) != null) {
57             System.out.println(line);
58         }
59     } catch (IOException e) {
60         System.out.println("Error while reading file: " + e.getMessage());
61     }
62 }
63
64 }
65
66 }
67

```

```
Account.java  CurrentAccou...  Bank.java  FileManager...  Main.java  *README.md  module-
1 package bankmanagement;
2
3 import java.util.Scanner;
4
5 public class Main {
6
7     private static Scanner scanner = new Scanner(System.in);
8     private static Bank bank = new Bank();
9
10    public static void main(String[] args) {
11
12        int choice;
13
14        do {
15            displayMenu();
16            choice = readInt("Enter your choice: ");
17
18            try {
19
20                switch (choice) {
21
22                    case 1:
23                        addAccount();
24                        break;
25
26                    case 2:
27                        bank.viewAllAccounts();
28                        break;
29
30                    case 3:
31                        searchAccount();
32                        break;
33
34                    case 4:
35                        updateAccount();
36                        break;
37
38                    case 5:
39                        deleteAccount();
40                        break;
41
42                    case 6:
43                        depositMoney();
44                        break;
45
```

```
Account.java  CurrentAccou...  Bank.java  FileManager...  Main.java ×  *README.md  module-info...

46         case 7:
47             withdrawMoney();
48             break;
49
50         case 8:
51             FileManager.saveAccounts(bank);
52             System.out.println("Thank you for using Bank Management System!");
53             break;
54
55         case 9:
56             FileManager.readAccounts();
57             break;
58
59         default:
60             System.out.println("Invalid choice. Please try again.");
61     }
62
63     } catch (IllegalArgumentException e) {
64         System.out.println("Error: " + e.getMessage());
65     }
66
67     } while (choice != 8);
68
69     scanner.close();
70 }
71
72 // Display menu
73 private static void displayMenu() {
74
75     System.out.println("\n=====");
76     System.out.println("      BANK MANAGEMENT SYSTEM");
77     System.out.println("=====");
78     System.out.println("1. Add Account");
79     System.out.println("2. View All Accounts");
80     System.out.println("3. Search Account");
81     System.out.println("4. Update Account");
82     System.out.println("5. Delete Account");
83     System.out.println("6. Deposit Money");
84     System.out.println("7. Withdraw Money");
85     System.out.println("8. Save & Exit");
86     System.out.println("9. Read Saved Data");
87     System.out.println("=====");
88 }
89
90 // Add account
```

```
Account.java  CurrentAccou...  Bank.java  FileManager...  Main.java  *README.m
91 private static void addAccount() {
92
93     int accountNumber = readInt("Enter Account Number: ");
94
95     System.out.print("Enter Account Holder Name: ");
96     String name = scanner.nextLine();
97
98     double balance = readDouble("Enter Initial Balance: ");
99
100    System.out.println("\nSelect Account Type:");
101    System.out.println("1. Savings Account");
102    System.out.println("2. Current Account");
103
104    int type = readInt("Enter type: ");
105
106    Account account;
107
108    if (type == 1) {
109
110        double interestRate =
111            readDouble("Enter Interest Rate (%): ");
112
113        account = new SavingsAccount(
114            accountNumber,
115            name,
116            balance,
117            interestRate
118        );
119
120    } else if (type == 2) {
121
122        double overdraftLimit =
123            readDouble("Enter Overdraft Limit: ");
124
125        account = new CurrentAccount(
126            accountNumber,
127            name,
128            balance,
129            overdraftLimit
130        );
131
132    } else {
133        System.out.println("Invalid account type.");
134        return;
135    }
}
```

```
Account.java  CurrentAccou...  Bank.java  FileManager...  Main.java  *README.md  m
133         System.out.println("Invalid account type.");
134         return;
135     }
136
137     bank.addAccount(account);
138 }
139
140 // Search account
141 private static void searchAccount() {
142
143     int accountNumber =
144         readInt("Enter Account Number to Search: ");
145
146     Account account = bank.searchAccount(accountNumber);
147
148     if (account == null) {
149         System.out.println("Account not found.");
150     } else {
151         System.out.println("\nAccount Found:");
152         account.displayAccountDetails();
153     }
154 }
155
156 // Update account
157 private static void updateAccount() {
158
159     int accountNumber =
160         readInt("Enter Account Number: ");
161
162     System.out.print("Enter New Account Holder Name: ");
163     String newName = scanner.nextLine();
164
165     bank.updateAccount(accountNumber, newName);
166 }
167
168 // Delete account
169 private static void deleteAccount() {
170
171     int accountNumber =
172         readInt("Enter Account Number to Delete: ");
173
174     bank.deleteAccount(accountNumber);
175 }
176
177 // Deposit
```

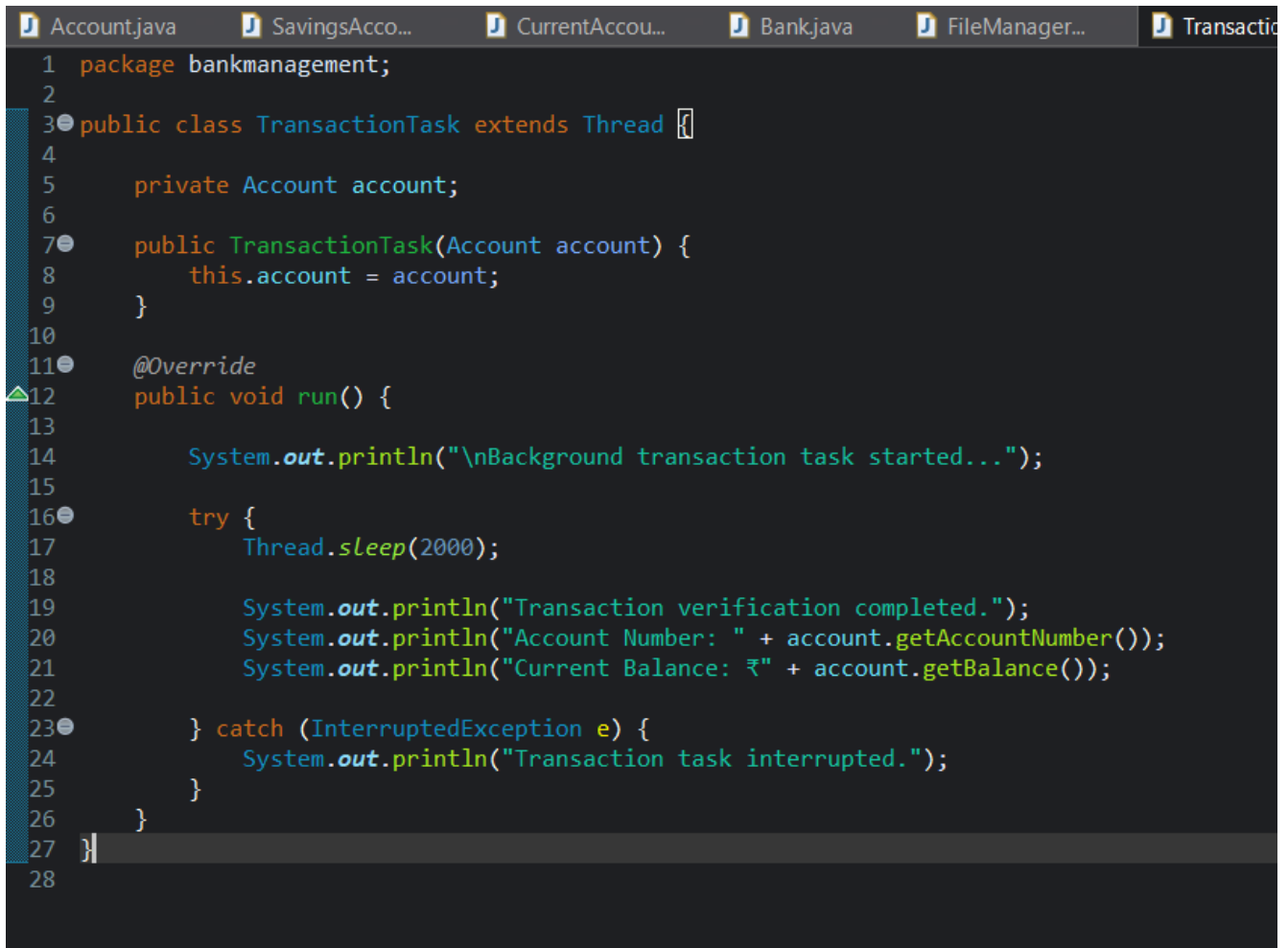
```
Account.java  CurrentAccou...  Bank.java  FileManager...  Main.java ×  *README.md

178 private static void depositMoney() {
179
180     int accountNumber =
181         readInt("Enter Account Number: ");
182
183     Account account = bank.searchAccount(accountNumber);
184
185     if (account == null) {
186         System.out.println("Account not found.");
187         return;
188     }
189
190     double amount =
191         readDouble("Enter Deposit Amount: ");
192
193     Transaction.deposit(account, amount);
194
195     startTransactionThread(account);
196 }
197
198 // Withdraw
199 private static void withdrawMoney() {
200
201     int accountNumber =
202         readInt("Enter Account Number: ");
203
204     Account account = bank.searchAccount(accountNumber);
205
206     if (account == null) {
207         System.out.println("Account not found.");
208         return;
209     }
210
211     double amount =
212         readDouble("Enter Withdrawal Amount: ");
213
214     Transaction.withdraw(account, amount);
215
216     startTransactionThread(account);
217 }
218
219 // Start background transaction verification
220 private static void startTransactionThread(Account account) {
221
222     TransactionTask task =
```

```
Account.java × CurrentAccou... Bank.java FileManager... Main.java × *README.md
222     TransactionTask task =
223         new TransactionTask(account);
224
225     task.start();
226 }
227
228 // Read integer safely
229 private static int readInt(String message) {
230
231     while (true) {
232
233         try {
234
235             System.out.print(message);
236             return Integer.parseInt(scanner.nextLine());
237
238         } catch (NumberFormatException e) {
239
240             System.out.println(
241                 "Please enter a valid number."
242             );
243         }
244     }
245 }
246
247 // Read double safely
248 private static double readDouble(String message) {
249
250     while (true) {
251
252         try {
253
254             System.out.print(message);
255             return Double.parseDouble(scanner.nextLine());
256
257         } catch (NumberFormatException e) {
258
259             System.out.println(
260                 "Please enter a valid amount."
261             );
262         }
263     }
264 }
265 }
266
```

```
Account.java SavingsAccount.java CurrentAccount.java Bank.java FileManager.java
1 package bankmanagement;
2
3 public class SavingsAccount extends Account {
4
5     private double interestRate;
6
7     // Constructor
8     public SavingsAccount(int accountNumber, String accountHolderName,
9                           double balance, double interestRate) {
10
11         super(accountNumber, accountHolderName, balance);
12         this.interestRate = interestRate;
13     }
14
15     // Getter
16     public double getInterestRate() {
17         return interestRate;
18     }
19
20     // Polymorphism
21     @Override
22     public String getAccountType() {
23         return "Savings Account";
24     }
25
26     // Calculate interest
27     public double calculateInterest() {
28         return balance * interestRate / 100;
29     }
30 }
```

```
Account.java SavingsAcco... × CurrentAccou... Bank.java FileManager... Main.java
1 package bankmanagement;
2
3 public class SavingsAccount extends Account {
4
5     private double interestRate;
6
7     // Constructor
8     public SavingsAccount(int accountNumber, String accountHolderName,
9                           double balance, double interestRate) {
10
11         super(accountNumber, accountHolderName, balance);
12         this.interestRate = interestRate;
13     }
14
15     // Getter
16     public double getInterestRate() {
17         return interestRate;
18     }
19
20     // Polymorphism
21     @Override
22     public String getAccountType() {
23         return "Savings Account";
24     }
25
26     // Calculate interest
27     public double calculateInterest() {
28         return balance * interestRate / 100;
29     }
30 }
```



```
1 package bankmanagement;
2
3 public class TransactionTask extends Thread {
4
5     private Account account;
6
7     public TransactionTask(Account account) {
8         this.account = account;
9     }
10
11     @Override
12     public void run() {
13
14         System.out.println("\nBackground transaction task started...");
15
16         try {
17             Thread.sleep(2000);
18
19             System.out.println("Transaction verification completed.");
20             System.out.println("Account Number: " + account.getAccountNumber());
21             System.out.println("Current Balance: ₹" + account.getBalance());
22
23         } catch (InterruptedException e) {
24             System.out.println("Transaction task interrupted.");
25         }
26     }
27 }
28
```

8. Output Screenshots

Insert screenshots of the actual Eclipse console output demonstrating the major features of the application.

6. Main menu and Add Account
7. View All Accounts
8. Search and Update Account
9. Delete Account
10. Deposit and Withdrawal
11. File Handling / Saved Data
12. Multithreading / Transaction Verification

```
Console X [Icons]
<terminated> Main (2) [Java Application] C:\Users\PANDU\p2\pool\plugins\org.ec

=====
      BANK MANAGEMENT SYSTEM
=====
1. Add Account
2. View All Accounts
3. Search Account
4. Update Account
5. Delete Account
6. Deposit Money
7. Withdraw Money
8. Save & Exit
9. Read Saved Data
=====
Enter your choice: 1
Enter Account Number: 101
Enter Account Holder Name: Bhavana
Enter Initial Balance: 10000

Select Account Type:
1. Savings Account
2. Current Account
Enter type: 1
Enter Interest Rate (%): 5
Account added successfully.

=====
      BANK MANAGEMENT SYSTEM
=====
1. Add Account
2. View All Accounts
3. Search Account
4. Update Account
5. Delete Account
6. Deposit Money
7. Withdraw Money
8. Save & Exit
9. Read Saved Data
=====
Enter your choice: 1
Enter Account Number: 102
Enter Account Holder Name: Ramya
Enter Initial Balance: 52000

Select Account Type:
1. Savings Account
2. Current Account
Enter type: 1
Enter Interest Rate (%): 1
Account added successfully.
```

```

=====
BANK MANAGEMENT SYSTEM
=====
1. Add Account
2. View All Accounts
3. Search Account
4. Update Account
5. Delete Account
6. Deposit Money
7. Withdraw Money
8. Save & Exit
9. Read Saved Data
=====
Enter your choice: 2

===== ALL ACCOUNTS =====
Account Number : 101
Account Holder : Bhavana
Account Type   : Savings Account
Balance       : ₹10000.0
-----
Account Number : 102
Account Holder : Ramya
Account Type   : Savings Account
Balance       : ₹52000.0
-----

```

```

=====
BANK MANAGEMENT SYSTEM
=====
1. Add Account
2. View All Accounts
3. Search Account
4. Update Account
5. Delete Account
6. Deposit Money
7. Withdraw Money
8. Save & Exit
9. Read Saved Data
=====
Enter your choice: 3
Enter Account Number to Search: 102

Account Found:
Account Number : 102
Account Holder : Ramya
Account Type   : Savings Account
Balance       : ₹52000.0

```

```

=====
      BANK MANAGEMENT SYSTEM
=====
1. Add Account
2. View All Accounts
3. Search Account
4. Update Account
5. Delete Account
6. Deposit Money
7. Withdraw Money
8. Save & Exit
9. Read Saved Data
=====
Enter your choice: 1
Enter Account Number: 103
Enter Account Holder Name: ramu
Enter Initial Balance: 82000

Select Account Type:
1. Savings Account
2. Current Account
Enter type: 1
Enter Interest Rate (%): 2
Account added successfully.

```

```

=====
      BANK MANAGEMENT SYSTEM
=====
1. Add Account
2. View All Accounts
3. Search Account
4. Update Account
5. Delete Account
6. Deposit Money
7. Withdraw Money
8. Save & Exit
9. Read Saved Data
=====
Enter your choice: 4
Enter Account Number: 102
Enter New Account Holder Name: ravi
Account updated successfully.

```

```
=====
      BANK MANAGEMENT SYSTEM
=====
1. Add Account
2. View All Accounts
3. Search Account
4. Update Account
5. Delete Account
6. Deposit Money
7. Withdraw Money
8. Save & Exit
9. Read Saved Data
=====
Enter your choice: 5
Enter Account Number to Delete: 103
Account deleted successfully.
```

```
=====
      BANK MANAGEMENT SYSTEM
=====
1. Add Account
2. View All Accounts
3. Search Account
4. Update Account
5. Delete Account
6. Deposit Money
7. Withdraw Money
8. Save & Exit
9. Read Saved Data
=====
Enter your choice: 6
Enter Account Number: 102
Enter Deposit Amount: 5000
₹5000.0 deposited successfully.
New Balance: ₹57000.0
```

```

=====
      BANK MANAGEMENT SYSTEM
=====
1. Add Account
2. View All Accounts
3. Search Account
4. Update Account
5. Delete Account
6. Deposit Money
7. Withdraw Money
8. Save & Exit
9. Read Saved Data
=====
Enter your choice:
Background transaction task started...
Transaction verification completed.
Account Number: 102
Current Balance: ₹57000.0

Please enter a valid number.
Enter your choice: 5
Enter Account Number to Delete: 3
Account not found.

=====
      BANK MANAGEMENT SYSTEM
=====
1. Add Account
2. View All Accounts
3. Search Account
4. Update Account
5. Delete Account
6. Deposit Money
7. Withdraw Money
8. Save & Exit
9. Read Saved Data
=====
Enter your choice: 7
Enter Account Number: 101
Enter Withdrawal Amount: 1000
₹1000.0 withdrawn successfully.
New Balance: ₹9000.0

```

```
=====
      BANK MANAGEMENT SYSTEM
=====
1. Add Account
2. View All Accounts
3. Search Account
4. Update Account
5. Delete Account
6. Deposit Money
7. Withdraw Money
8. Save & Exit
9. Read Saved Data
=====
Enter your choice:
Background transaction task started...
Transaction verification completed.
Account Number: 101
Current Balance: ₹9000.0

Please enter a valid number.
Enter your choice: 8
Accounts saved to file successfully.
Thank you for using Bank Management System!
```

9. Challenges Faced

Understanding OOP Design: It was initially difficult to decide how different account types should be represented. An abstract Account class was created, with SavingsAccount and CurrentAccount as subclasses.

Managing Multiple Accounts: Managing multiple account records using normal variables would be difficult. A HashMap<Integer, Account> was used to store accounts using account number as the key.

Handling Invalid Input: Invalid numbers and transaction amounts could cause errors. try-catch blocks and input validation were implemented.

File Handling: Saving and reading account information required proper exception handling. FileWriter, PrintWriter, FileReader, and BufferedReader were used.

Multithreading: Understanding how to execute a supporting task in a separate thread was challenging. TransactionTask was created by extending Thread to perform background transaction verification.

10. Conclusion

The Bank Account Management System successfully demonstrates the practical application of Java programming concepts learned during the internship. The project provided hands-on experience with Object-Oriented Programming, inheritance, polymorphism, abstraction, encapsulation, collections, exception handling, file handling, and multithreading. It also helped improve practical skills in Eclipse, Git, GitHub, project organization, and documentation. Through this project, I gained a better understanding of how Java concepts can be combined to develop a functional real-world application.